Memory in javaScript

JavaScript memory is mainly divided into two parts:

1️⃣ Stack Memory

- **Stores primitive values**
- **Stores function calls (execution context)**
- **Very fast**
- **Automatically managed**

👉 Primitives are copied by value.

When a function runs:

1. **Engine creates an Execution Context**
2. **Pushes it onto the stack**
3. **When function finishes → popped off**

If stack exceeds limit → Stack Overflow

2️⃣ Heap Memory

- **Stores objects and reference types**
- **Larger, dynamic memory**
- **Garbage collected automatically**

👉 Objects are copied by reference (both point to same memory).

3️⃣ Garbage Collection

JavaScript uses automatic garbage collection.

When an object is:

- **Not referenced anymore**

- **Not reachable**

It gets removed from memory automatically.

Modern engines like V8 (Chrome, Node.js) use:

- **Mark-and-sweep algorithm**

  Step 1 — Mark

  Start from Root

  - **Global object**
  - **Current call stack**

  Mark everything reachable

  Step 2 — Sweep

  Remove everything NOT reachable

  Example:

- **Generational garbage collection**

  1. Young Generation

  - **Short-lived objects**
  - **Frequently cleaned**
  - **Fast GC**

  2. Old Generation

  - **Long-lived objects**
  - **Cleaned less often**
  - **More expensive GC**

  Most objects die young → so cleaning young space frequently improves performance.

6️⃣ Memory Leaks in JS

Memory leak = object stays in memory but not needed.

❌ Global Variables

```
leak = "I am global";
```

❌ Forgotten Timers

```
setInterval(() => {
  console.log("running");
}, 1000);
```

❌ Closures Holding References

```
function outer() {
  let bigData = new Array(1000000).fill("*");

  return function inner() {
    console.log("Hello");
  };
}
```

Even if inner doesn't use bigData, it's still retained.

7️⃣ Closures & Memory

Closures store variables in Lexical Environment, which lives in heap.

8️⃣ Shallow vs Deep Copy (Memory Impact)

Shallow Copy

Copies reference

```
let a = { x: 1 };
let b = { ...a };
```

Nested objects still reference same memory.

Deep Copy

Creates new memory allocation

```
let deep = structuredClone(a);
```